1. Document Purpose

This document defines the technical requirements for a web-based **Study and Task Management Application**.

The application allows users to:

- Define a STUDY and its attributes.
- Create TASKS for the STUDY.
- Create TASKS from a predefined STUDY TYPE template.
- Modify TASKS imported from a template.
- Create TASKS from scratch.
- Save a modified TASK set as a new STUDY TYPE.
- Track TASK status.
- Track current and historical TASK dates.
- Report progress and expected completion across all ACTIVE STUDIES.

This specification defines:

- Data model
- Business logic
- Application screens
- User interactions
- Validation rules
- APIs
- Reporting logic
- Audit requirements
- Security and permissions
- Non-functional requirements
- Acceptance criteria

---

2. Core Domain Model

The application has four primary concepts.

2.1 STUDY TYPE

A reusable definition of a workflow.

A STUDY TYPE contains:

- Name
- Description
- Ordered list of TASK definitions

A STUDY TYPE is a template only.

It does not contain execution information such as:

- Actual status
- Current dates
- Historical dates
- Completion dates

2.2 STUDY

An actual instance of work.

A STUDY contains:

- Study-level attributes
- TASKS
- Overall status
- Planned dates
- Calculated expected completion
- Progress indicators

A STUDY may originate from:

- An existing STUDY TYPE
- An existing STUDY TYPE that has subsequently been modified
- A completely custom set of TASKS

2.3 TASK

An individual piece of work belonging to one STUDY.

Each TASK contains:

- Name
- Description
- Sequence
- Start date
- End date
- Status
- Date history
- Status history

2.4 TASK HISTORY

Historical records must be immutable.

The system must separately track:

- TASK date changes
- TASK status changes

Historical records may be read but not edited.

---

3. User Roles

Version 1 should support at least two roles.

3.1 Standard User

Can:

- View STUDIES
- Create STUDIES
- Edit STUDIES they have permission to access
- Create/edit TASKS
- Change TASK dates
- Change TASK status
- View TASK history
- View dashboards
- Create a STUDY TYPE from a STUDY if permitted

3.2 Administrator

Can perform all Standard User actions plus:

- Create STUDY TYPES
- Edit STUDY TYPES
- Archive STUDY TYPES
- Manage users
- Manage application-level configuration

The permission structure should be extensible to additional roles later.

---

4. STUDY Data Definition

4.1 STUDY Table

Suggested database table:

studies

Fields:

Field	Type	Required	Notes
id	UUID / BIGINT	Yes	Primary key
study_name	VARCHAR(255)	Yes	User entered
study_type_id	FK	No	Null for completely custom studies
description	TEXT	No	Study description
owner_user_id	FK	Yes	Study owner
client_business_unit	VARCHAR(255)	No	Configurable
priority	ENUM	No	LOW / MEDIUM / HIGH
original_start_date	DATE	No	Planned at creation
original_completion_date	DATE	No	Planned at creation
current_expected_completion_date	DATE	No	Calculated
status	ENUM	Yes	DRAFT / ACTIVE / COMPLETE / CANCELLED
created_by	FK	Yes	User ID
created_at	TIMESTAMP	Yes	System generated
updated_at	TIMESTAMP	Yes	System generated
completed_at	TIMESTAMP	No	Populated when complete
archived_at	TIMESTAMP	No	If soft delete/archive used

The Study fields should be implemented so additional study parameters can be added later without major redesign.

---

5. STUDY TYPE Data Definition

5.1 STUDY TYPE Table

study_types

Field	Type	Required
id	UUID / BIGINT	Yes
name	VARCHAR(255)	Yes
description	TEXT	No
status	ENUM	Yes
created_by	FK	Yes
created_at	TIMESTAMP	Yes
updated_at	TIMESTAMP	Yes
archived_at	TIMESTAMP	No

Status values:

- ACTIVE
- ARCHIVED

Archived STUDY TYPES cannot be selected for new STUDIES.

Existing STUDIES created from the archived STUDY TYPE remain unaffected.

---

6. STUDY TYPE TASK Data Definition

Table:

study_type_tasks

Field	Type	Required
id	UUID / BIGINT	Yes
study_type_id	FK	Yes
task_name	VARCHAR(255)	Yes
description	TEXT	No
sequence_number	INTEGER	Yes
default_duration_days	INTEGER	No
created_at	TIMESTAMP	Yes
updated_at	TIMESTAMP	Yes

sequence_number determines display order.

Sequence must be unique within a STUDY TYPE.

---

7. TASK Data Definition

Table:

tasks

Field	Type	Required
id	UUID / BIGINT	Yes
study_id	FK	Yes
source_study_type_task_id	FK	No
task_name	VARCHAR(255)	Yes
description	TEXT	No
sequence_number	INTEGER	Yes
current_start_date	DATE	Yes
current_end_date	DATE	Yes
status	ENUM	Yes
date_change_count	INTEGER	Yes
created_by	FK	Yes
created_at	TIMESTAMP	Yes
updated_at	TIMESTAMP	Yes
completed_at	TIMESTAMP	No

Task status values:

- NOT_STARTED
- IN_PROGRESS
- COMPLETE

Default:

NOT_STARTED

Default date_change_count: 0

---

8. TASK Date History

Table:

task_date_history

Each save operation that changes either TASK start date or TASK end date creates one history event.

Field	Type	Required
id	UUID / BIGINT	Yes
task_id	FK	Yes
previous_start_date	DATE	Yes
previous_end_date	DATE	Yes
new_start_date	DATE	Yes
new_end_date	DATE	Yes
version_number	INTEGER	Yes
changed_by	FK	Yes
changed_at	TIMESTAMP	Yes

Important rule:

If both start and end date are changed in one save operation, this counts as **one date change event**.

Example:

Before:

2026-10-01 → 2026-10-05

After:

2026-10-03 → 2026-10-08

Result:

date_change_count = date_change_count + 1

Not +2.

---

9. TASK Status History

Table:

task_status_history

Field	Type	Required
id	UUID / BIGINT	Yes
task_id	FK	Yes
previous_status	ENUM	Yes
new_status	ENUM	Yes
changed_by	FK	Yes
changed_at	TIMESTAMP	Yes

Every actual status change creates one record.

Saving a TASK without changing status must not create a status history record.

---

10. General Audit Log

A generic audit log is also recommended.

Table:

audit_log

Fields:

- id
- entity_type
- entity_id
- action
- user_id
- previous_value JSON
- new_value JSON
- timestamp

Examples of action:

- STUDY_CREATED
- STUDY_UPDATED
- STUDY_STATUS_CHANGED
- TASK_CREATED
- TASK_UPDATED
- TASK_DELETED
- TASK_DATE_CHANGED
- TASK_STATUS_CHANGED
- STUDY_TYPE_CREATED
- STUDY_TYPE_UPDATED

Audit records are immutable.

---

11. STUDY Creation Workflow

The Create STUDY workflow contains two stages.

Stage 1 — Define STUDY

User enters Study attributes.

Minimum required fields:

- Study Name
- Owner
- Study Status
- Original Start Date
- Original Completion Date

Additional business fields may be configured.

User selects:

How would you like to create the Study Tasks?

Options:

1. Use an existing STUDY TYPE without modification
2. Start with an existing STUDY TYPE and modify it
3. Build TASKS from scratch

---

12. Workflow 1 — Use STUDY TYPE As-Is

User:

1. Selects STUDY TYPE.
2. Reviews TASK list.
3. Confirms creation.

System:

1. Creates STUDY.
2. Copies each STUDY TYPE TASK into tasks.
3. Assigns each copied TASK to new STUDY.
4. Sets status to NOT_STARTED.
5. Sets date change count to 0.
6. Sets initial TASK dates.
7. Stores source STUDY TYPE ID.

After copying, the TASK records are independent.

Later changes to the template do not affect the STUDY.

---

13. Workflow 2 — Use and Modify STUDY TYPE

User:

1. Selects STUDY TYPE.
2. System loads template TASKS into an editable working list.
3. User can:
    - Add TASKS
    - Delete TASKS
    - Rename TASKS
    - Edit TASK descriptions
    - Reorder TASKS
    - Define dates
4. User saves STUDY.

Optional action:

Save this TASK structure as a new STUDY TYPE

If selected:

User must enter:

- New STUDY TYPE Name
- Description, optional

System creates:

- New STUDY TYPE
- New STUDY TYPE TASK records

Execution-specific information must not be copied into the new template.

---

14. Workflow 3 — Build From Scratch

User:

1. Selects "Build Tasks From Scratch."
2. Adds TASKS.

3. Defines:
    - Task Name
    - Description
    - Start Date
    - End Date
    - Sequence
4. Creates STUDY.

Optional:

Save Task List as New Study Type

Same rules as Workflow 2 apply.

---

15. TASK Creation Rules

When a TASK is created:

status = NOT_STARTED

date_change_count = 0

created_at = current timestamp

No TASK date history record is required for initial creation.

The initial TASK dates are treated as the original dates.

Alternative implementation is acceptable if the original dates are explicitly stored as history version 0.

---

16. TASK Date Editing

When a user edits a TASK:

System compares:

- Existing start date
- Existing end date

with:

- Submitted start date
- Submitted end date

If neither changes:

Do not:

- Create history record
- Increment date change count

If either changes:

System must:

1. Create task_date_history record.
2. Store previous dates.
3. Store new dates.
4. Increment date_change_count by 1.
5. Update TASK's current dates.
6. Recalculate STUDY expected completion date.
7. Update study dashboard calculations.

All steps should execute within one database transaction.

---

17. TASK Date Validation

Required validation:

end_date >= start_date

The application must reject:

End Date < Start Date

with a clear message.

Example:

"Task End Date cannot be earlier than Task Start Date."

---

18. TASK Status Rules

Valid transitions:

NOT_STARTED → IN_PROGRESS

NOT_STARTED → COMPLETE

IN_PROGRESS → COMPLETE

The application may also allow backwards transitions:

COMPLETE → IN_PROGRESS

IN_PROGRESS → NOT_STARTED

This should be permitted in Version 1 unless business rules later prohibit it.

Every transition must be recorded.

When TASK becomes COMPLETE:

Set:

completed_at = current timestamp

If TASK is subsequently reopened:

Set:

completed_at = null

or maintain completion events separately in history.

---

19. STUDY Status

Allowed values:

- DRAFT
- ACTIVE
- COMPLETE
- CANCELLED

DRAFT

Study definition is incomplete or not yet started.

ACTIVE

Study is operational and appears on the Active Studies Dashboard.

COMPLETE

Study has concluded.

CANCELLED

Study was discontinued.

---

20. STUDY Status Logic

The system should not automatically mark a STUDY complete unless explicitly configured.

Recommended Version 1 behaviour:

When all TASKS become COMPLETE:

Display:

"All tasks are complete. Mark this Study as Complete?"

User confirms.

This prevents unintended automatic closure.

---

21. Expected Study Completion

For ACTIVE STUDIES:

Current Expected Completion Date = latest current_end_date among all TASKS not marked COMPLETE

If all TASKS are COMPLETE:

Use either:

- Date of final TASK completion

or

- Last current TASK end date

Recommended:

Use final TASK completion date for actual completion reporting.

---

22. Original Completion Date

The STUDY's:

original_completion_date

must remain unchanged after activation.

This is the baseline schedule.

Users may modify the current task schedule without changing the baseline.

If administrators need to change the baseline in future, that should be a separately audited operation.

---

23. Schedule Variance

Calculate:

schedule_variance_days = current_expected_completion_date - original_completion_date

Examples:

0:

On Schedule

+5:

5 Days Late

-3:

3 Days Ahead

This value should be dynamically calculated rather than permanently stored unless required for performance.

---

24. Study Progress Calculation

Version 1:

Progress % = Complete Tasks / Total Tasks × 100

Example:

6 complete tasks from 10:

60%

If Study has zero Tasks:

Progress = 0%.

---

25. Task Status Summary

For each STUDY calculate:

- Total TASKS
- NOT_STARTED count
- IN_PROGRESS count
- COMPLETE count

Example:

10 Tasks | 3 Not Started | 2 In Progress | 5 Complete

---

26. Current Study Stage

Default algorithm:

1. Sort TASKS by sequence_number.
2. Find first TASK where status != COMPLETE.
3. Return its TASK Name.

If every TASK is COMPLETE:

Current Stage = Complete

Potential future enhancement:

Support explicit Study phases or concurrent stages.

---

27. Dashboard Requirements

Route:

/dashboard

Show all STUDIES where:

status = ACTIVE

Recommended columns:

- Study Name
- Study Type
- Owner
- Study Start Date
- Progress %
- Completed / Total TASKS
- Current Stage
- Current Expected Completion
- Original Completion
- Schedule Variance
- Last Updated

---

28. Dashboard Filters

Minimum filters:

- Owner
- Study Type
- Study Status
- Expected Completion Date
- Priority

Additional filters may include:

- Client / Business Unit
- Late studies only
- Studies completing in next X days

---

29. Dashboard Sorting

Allow sorting by:

- Study Name
- Expected Completion
- Schedule Variance
- Progress %
- Owner
- Priority
- Last Updated

---

30. STUDY List Screen

Route:

/studies

Shows:

- ACTIVE
- DRAFT

- COMPLETE
- CANCELLED

Default filter:

ACTIVE

Search by:

- Study Name
- Study ID
- Owner
- Study Type

---

31. STUDY Detail Screen

Route:

/studies/{study_id}

Page sections:

Study Header

Show:

- Study Name
- Status
- Owner
- Study Type
- Priority

Actions:

- Edit Study
- Change Status
- Add Task
- Save Task Structure as Study Type

Study Summary

Show:

- Original Start Date
- Original Completion Date
- Current Expected Completion
- Schedule Variance
- Progress %
- Task count

Task Table

Columns:

Field
Sequence
Task Name
Status
Current Start
Current End
Previous Start
Previous End
Date Change Count
Actions

Actions:

- Edit
- Change Status
- View History
- Delete

---

32. TASK Detail Screen

Route:

/tasks/{task_id}

Display:

- Task Name
- Study
- Description
- Sequence
- Status
- Current Start Date
- Current End Date
- Previous Start Date
- Previous End Date
- Date Change Count
- Completed Date

Sections:

Date History

Show newest first.

Columns:

- Version
- Previous Start
- Previous End
- New Start
- New End
- Changed By
- Changed At

Status History

Columns:

- Previous Status
- New Status
- Changed By

- Changed At

---

33. Previous Date Logic

For TASK display:

Previous Start Date and Previous End Date should come from the most recent task_date_history record.

If no changes have occurred:

Previous Dates = blank / "No previous dates"

Date Change Count = 0

---

34. STUDY TYPE Management

Route:

/study-types

List:

- Study Type Name
- Description
- Number of Tasks
- Status
- Created Date
- Last Updated

Actions:

- View
- Edit
- Duplicate
- Archive

---

35. STUDY TYPE Edit Screen

Route:

/study-types/{study_type_id}

User can:

- Change name
- Change description
- Add TASK
- Delete TASK
- Rename TASK
- Edit description
- Reorder TASKS

Saving changes affects future STUDIES only.

Existing STUDIES must remain unchanged.

---

36. TASK Reordering

TASK sequence should support drag-and-drop.

On save:

Sequence numbers should be normalized:

1, 2, 3, 4...

No gaps are required.

Reordering TASKS should not count as a TASK date change.

---

37. TASK Deletion

For DRAFT STUDIES:

TASK deletion can be hard delete or soft delete.

For ACTIVE STUDIES:

Recommended:

Use soft delete or audit record.

System must record:

- Deleted TASK
- User
- Timestamp

TASK history should remain available for audit.

---

38. Suggested Soft Delete Pattern

For TASKS add:

deleted_at

deleted_by

Queries should normally exclude records where:

deleted_at IS NOT NULL

---

39. API Design

Recommended style:

REST JSON API.

Base:

/api/v1

---

40. STUDY API

List Studies

GET /api/v1/studies

Optional parameters:

- status
- owner_id
- study_type_id
- priority
- completion_from
- completion_to
- search
- page
- page_size
- sort

Get Study

GET /api/v1/studies/{id}

Returns:

- Study attributes
- Calculated metrics
- Tasks

Create Study

POST /api/v1/studies

Example payload:


```
{
  "study_name": "Q4 Concept Test",
  "owner_user_id": "123",
  "study_type_id": "456",
  "description": "Testing Q4 concepts",
  "original_start_date": "2026-10-01",
  "original_completion_date": "2026-10-30",
  "priority": "HIGH",
  "status": "DRAFT",
  "task_creation_method": "MODIFIED_TEMPLATE"
}
```

Update Study

PATCH /api/v1/studies/{id}

Change Study Status

POST /api/v1/studies/{id}/status

---

41. TASK API

Create Task

POST /api/v1/studies/{study_id}/tasks

Update Task

PATCH /api/v1/tasks/{task_id}

Change Task Status

POST /api/v1/tasks/{task_id}/status

Example:

```
{
  "status": "IN_PROGRESS"
}
```

Delete Task

DELETE /api/v1/tasks/{task_id}

Reorder Tasks

POST /api/v1/studies/{study_id}/tasks/reorder

Example:

```
{
  "tasks": [
    {"task_id": "101", "sequence": 1},
    {"task_id": "104", "sequence": 2},
    {"task_id": "102", "sequence": 3}
  ]
}
```

---

42. TASK History API

Get Date History

GET /api/v1/tasks/{task_id}/date-history

Get Status History

GET /api/v1/tasks/{task_id}/status-history

---

43. STUDY TYPE API

List

GET /api/v1/study-types

Create

POST /api/v1/study-types

Retrieve

GET /api/v1/study-types/{id}

Update

PATCH /api/v1/study-types/{id}

Archive

POST /api/v1/study-types/{id}/archive

Duplicate

POST /api/v1/study-types/{id}/duplicate

---

44. Save STUDY as STUDY TYPE API

Endpoint:

POST /api/v1/studies/{study_id}/create-study-type

Payload:

```
{
  "name": "Modified Concept Test",
  "description": "Concept test workflow including stakeholder review"
}
```

System copies:

- Task Name

- Task Description
- Sequence

Do not copy:

- Dates
- Status
- History
- Completion information

---

45. Reporting API

Endpoint:

GET /api/v1/reports/active-studies

Return for each Study:

- Study ID
- Study Name
- Owner
- Study Type
- Progress %
- Total Tasks
- Completed Tasks
- In Progress Tasks
- Not Started Tasks
- Current Stage
- Original Completion
- Expected Completion
- Schedule Variance

---

46. Database Relationships

Relationship model:

STUDY_TYPE 1 → MANY STUDY_TYPE_TASK

STUDY_TYPE 1 → MANY STUDY

STUDY 1 → MANY TASK

TASK 1 → MANY TASK_DATE_HISTORY

TASK 1 → MANY TASK_STATUS_HISTORY

USER 1 → MANY STUDY

USER 1 → MANY AUDIT_LOG

---

47. Referential Integrity

Deleting a STUDY TYPE must not delete STUDIES created from it.

Therefore:

STUDY TYPE should normally be archived rather than deleted.

Deleting a STUDY should normally be implemented as archive/soft delete if auditability is important.

---

48. Transaction Requirements

The following operations must use database transactions.

TASK Date Change

Transaction includes:

- Insert date history
- Update current dates
- Increment change count
- Update STUDY expected completion

STUDY Creation From Template

Transaction includes:

- Create STUDY
- Copy all TASKS
- Set sequence
- Calculate dates
- Calculate expected completion

Save as New STUDY TYPE

Transaction includes:

- Create STUDY TYPE
- Copy TASK structure

Failure of any step must roll back the entire transaction.

---

49. Concurrency

Use optimistic concurrency control.

Recommended:

Add either:

version

or rely on:

updated_at

When a user attempts to update an outdated record:

Return:

409 Conflict

UI message:

"This item has been changed by another user. Refresh the page before saving your changes."

---

50. Authentication

Application must require authenticated users.

Recommended options:

- Microsoft Entra ID
- Google Workspace
- Auth0
- Internal username/password

Authentication implementation should remain separate from domain logic.

---

51. Authorization

Every API request must validate that the authenticated user is permitted to access or modify the requested record.

Client-side hiding of buttons is not sufficient.

Authorization must be enforced server-side.

---

52. Validation Rules

At minimum:

STUDY

- Study Name required
- Owner required
- Original Completion >= Original Start
- Status must be valid ENUM

TASK

- Task Name required
- Start Date required
- End Date required
- End Date >= Start Date
- Status must be valid ENUM
- Sequence must be positive integer

STUDY TYPE

- Name required
- Name should be unique among ACTIVE types

---

53. Error Handling

API should return standard HTTP responses.

Examples:

200 Successful request

201 Created

400 Validation error

401 Not authenticated

403 Not authorized

404 Record not found

409 Concurrent edit conflict

500 Unexpected server error

Validation errors should return field-specific messages.

---

54. User Interface Behaviour

The UI should provide immediate visual confirmation after successful save.

Examples:

- "Task updated."
- "Task dates changed."
- "Study created."
- "Study Type created."

Avoid browser-native confirmation dialogs where possible.

Use application modal dialogs.

---

55. Date Display

Database dates should use ISO format internally:

YYYY-MM-DD

UI display should use user locale.

Example:

17 Sep 2026

or

Sep 17, 2026

Timezone-sensitive timestamps should be stored in UTC.

---

56. Responsive Design

Application must work on:

- Desktop
- Tablet
- Mobile browser

Primary workflow can be desktop optimized.

Dashboard tables may use horizontal scrolling on smaller screens.

---

57. Performance Requirements

Target Version 1:

Normal page load:

< 2 seconds under expected operational load.

Typical API requests:

< 1 second.

Dashboard should support at least:

- 1,000 Studies
- 100 Tasks per Study
- Tens of thousands of TASK history records

without redesign.

---

58. Search Performance

Database indexes recommended on:

studies.status

studies.owner_user_id

studies.study_type_id

studies.current_expected_completion_date

tasks.study_id

tasks.status

task_date_history.task_id

task_status_history.task_id

---

59. Accessibility

Application should target WCAG 2.1 AA.

Minimum:

- Keyboard navigation
- Semantic labels
- Accessible form controls
- Visible focus state
- Sufficient contrast
- Status not communicated through colour alone

---

60. Security Requirements

Minimum:

- HTTPS
- Secure authentication
- Role-based access control
- Server-side authorization
- Protection against SQL injection
- Cross-Site Request Forgery protection where applicable
- Input sanitization
- Secure password handling if internal auth is used
- Audit records for significant changes

---

61. Backup and Recovery

Production database should support:

- Automated daily backup
- Point-in-time recovery where supported
- Retention policy defined operationally

Audit and history records must be included in backup.

---

62. MVP Acceptance Criteria

Study Creation

Given a user creates a new STUDY,

when all mandatory fields are valid,

then the system creates the STUDY successfully.

Template Creation

Given a user selects an existing STUDY TYPE,

when they create the STUDY,

then all template TASKS are copied into the new STUDY.

Changes made later to the STUDY TYPE must not alter those TASKS.

Modified Template

Given a user starts from a STUDY TYPE,

when they add, remove or reorder TASKS,

then only the new STUDY reflects those changes.

Save as Study Type

Given a user modifies a TASK list,

when they select "Save as New Study Type,"

then a new reusable STUDY TYPE is created.

The new STUDY TYPE must not contain dates, TASK status or historical data.

Task Default Status

Given a new TASK is created,

then its status is:

NOT_STARTED

Task Date Change

Given a TASK's current dates are:

October 1–October 5,

when the user changes them to October 3–October 8,

then:

- Current dates become October 3–October 8
- Previous dates show October 1–October 5
- Date Change Count increases by one
- A date history record is created

Multiple Date Changes

Given TASK dates have been modified three times,

then:

- Current dates show the latest dates
- Previous dates show the immediately preceding dates
- Date Change Count = 3
- Full history contains all three change events

Dual Date Change

Given both TASK start and end dates are modified during one save,

then Date Change Count increases by exactly one.

Status Change

Given TASK status is NOT_STARTED,

when user changes it to IN_PROGRESS,

then:

- TASK status becomes IN_PROGRESS
- Status history records the transition

Active Studies Dashboard

Given there are multiple ACTIVE STUDIES,

when user opens the dashboard,

then all ACTIVE STUDIES are displayed with:

- Progress
- Current stage
- Expected completion
- Schedule variance

Expected Completion

Given a STUDY contains incomplete TASKS,

then Current Expected Completion equals the latest End Date among incomplete TASKS.

Study Progress

Given 6 of 10 TASKS are COMPLETE,

then STUDY Progress = 60%.

---

63. Recommended MVP Screen Structure

Navigation:

Dashboard

Studies

Create Study

Study Types

Administration

Application flow:

Dashboard → Study → Task → History

and

Study Types → Template → Create Study

---

64. Recommended Phase 2 Capabilities

Do not include these in the initial MVP unless required.

Potential later capabilities:

- TASK dependencies
- Automatic date propagation
- Gantt charts
- Milestones
- Notifications
- Due-date reminders
- Comments
- Attachments
- Custom fields
- Approval workflows
- Team assignments
- Per-task ownership
- Slack / Teams integration
- Email notifications
- Study-level history
- Calendar integrations
- Workflow rules
- Portfolio analytics
- Risk indicators
- Baseline schedule comparison
- Forecasting based on historical Study performance

---

65. Architectural Principle

The application should maintain a strict separation between:

Template Definition

STUDY_TYPE + STUDY_TYPE_TASK

and

Execution

STUDY + TASK

and

Historical Record

TASK_DATE_HISTORY + TASK_STATUS_HISTORY

The template should determine how a STUDY begins.

Once the STUDY is created, its TASK structure becomes independent of the template.

This separation is critical to preserving historical accuracy, allowing customization, and supporting future workflow functionality.

---

66. Recommended Technical Architecture

The specification is technology-neutral, but a standard implementation could use:

Frontend

React / Next.js or equivalent SPA framework.

Backend

Node.js / TypeScript, Python / FastAPI, .NET, Java Spring or equivalent.

Database

PostgreSQL recommended.

API

REST JSON.

Authentication

Enterprise SSO or managed authentication provider.

Hosting

Any modern managed cloud platform.

The design should prioritize maintainability and conventional architecture rather than unnecessary microservices.

For the expected scope, a modular monolithic application with a relational database is sufficient.

---

67. Suggested Development Modules

The codebase should separate the following functional modules:

1. Authentication and Users
2. Study Management
3. Study Type Management
4. Task Management
5. Task History
6. Dashboard and Reporting
7. Audit
8. Administration

Each module should expose its business logic through application services rather than putting business rules directly into UI components.

---

68. Definition of MVP Completion

Version 1 is considered complete when a user can successfully:

1. Create a STUDY.
2. Start from a STUDY TYPE or create TASKS manually.
3. Modify the TASK structure.
4. Save a resulting structure as a new STUDY TYPE.
5. Assign TASK dates.
6. Change TASK dates repeatedly without losing history.
7. See current dates, previous dates and number of changes.
8. Move TASKS between the three statuses.
9. View complete TASK date and status history.
10. View all ACTIVE STUDIES in one dashboard.
11. See progress, current stage and expected completion for each STUDY.
12. Identify STUDIES whose expected completion has moved relative to their original schedule.